

HERE IS THE FUNCTIONAL REQUIREMENTS:

1. The user will register using his mail and create his profile, after which an he will be allowed to choose what type of account he wants to open(savings, current or both) and the account will be automatically created and be put into pending approval state. This account and details will move to the admin dashboard in the user service and the admin can approve or reject the account creation. When approved the account gets linked with the user email and notification is sent using kafka.
2. In the user-service, it will contain customer and admin dashboard based on the credentials during login and role. The admin will have the following privileges: Approval of account, Deposit Money into the customers account, Freeze unfreeze and block unblock the account, Check the number of customers, will contain the audit logs of every user transaction but the admin must require a passkey from the audit team to access the audit logs. The customer will have the following privileges: Can reset password(using forgot password), update their profile, Check their account status wether active/freezed/blocked.
3. In the Account-service, it will contain the following functionalities: Switch account type(If one savings account, then change it to current and vice versa), Every account as a four digit pin to check balance and confirm transaction. Able to check balance, Create a bank account in the first step during registration, Able to close account on the request of user.
4. In the Transaction-service, it will contain the following requirement: Able to transfer money from one account to another, Able to download account statement, Able to perform internal transfer between self account
5. In the notification-service, it will have the following requirements: Able to send an email using google smtp whenever an event has been published from kafka.
6. In the auth-service, it will contain the following requirements: User registration, User can login and logout, Password and MFA(OTP to email), JWT Token (Which will contain the role of customer or admin)

Are there any correction or suggestions from your side?

Yes. I would make a few important corrections before you start implementation. I also checked the **TransactSphere repository again** and will treat it as the reference baseline, while adapting it to your requirements. The reference repo uses OpenFeign for immediate-consistency operations and Kafka for asynchronous notifications, but it currently uses more than five services, PostgreSQL, React, and Kubernetes/CoreDNS rather than your Oracle/OJET/Podman stack.

1. Your functional architecture is good, but one boundary should change

Your five services should remain:

1. Auth Service
2. User Service
3. Account Service
4. Transaction Service
5. Notification Service

I would **not put the audit database/workflow inside User Service** just because the admin dashboard is there.

Instead:

User Service

└ Customer + Admin Dashboard/UI APIs

Audit responsibilities

└ Notification Service

However, the **admin dashboard can display audit information through the Notification/Audit capability**. The UI location does not determine data ownership.

Your reference TransactSphere separates audit and notification into their own concerns, which is a useful pattern to retain even though you're consolidating to five services.

2. Registration flow

Your requirement is:

User

↓

Register email

↓

Create profile

↓

Choose:

Savings

Current

Both

↓

Account(s) automatically created

↓

PENDING_APPROVAL

↓

Admin dashboard

↓

Approve / Reject

↓

Approved

↓

Account linked to user's digital identity

↓

Kafka

↓

Email notification

I think this is good.

But I would make **one important distinction**:

Registration does NOT mean the bank account is active.

You should have:

Customer Status:

PENDING_APPROVAL

and:

Account Status:

PENDING_APPROVAL

Then after approval:

Customer Status:

ACTIVE

Account Status:

ACTIVE

And only then should the account become visible/usable through the customer's netbanking access.

3. "Both" should mean two accounts

You currently say:

Savings, current or both

I recommend defining this explicitly:

Savings

→ 1 savings account

Current

→ 1 current account

Both

→ 1 savings account

+ 1 current account

Don't create one account with:

ACCOUNT_TYPE = BOTH

Instead:

CUSTOMER 1001

ACCOUNT 5001 → SAVINGS

ACCOUNT 5002 → CURRENT

This will make your Account Service and Transaction Service much cleaner.

4. Account approval should belong to Account Service

You said:

account and details will move to admin dashboard in user service and admin can approve or reject account creation

The **dashboard can absolutely be in User Service**, but I would architect the operation like this:

Admin OJET

|

v

User Service

|

| OpenFeign

v

Account Service

|

+--> approveAccount()

|

+--> ACCOUNT = ACTIVE

|

v

Kafka

|

+--> Notification

+--> Audit

Why?

Because:

Account Service must remain the owner of account state.

User Service should not directly update:

ACCOUNT.STATUS

That would violate the service boundary.

5. The same applies to admin deposits

You said:

Admin can deposit money into the customer's account.

I would **not implement the deposit logic in User Service.**

Instead:

Admin Dashboard

|

v

User Service

|

| OpenFeign

v

Account Service

|

+--> validate account

+--> credit balance

+--> create ledger entry

|

v

Kafka

|

+--> Notification

+--> Audit

The UI is in User Service.

The **money operation belongs to Account Service.**

This is one of the most important design rules for your project.

6. Freeze/block/unfreeze/unblock

These should also be owned by Account Service.

I would define the meanings clearly because otherwise your team will implement them differently.

Account status

PENDING_APPROVAL

ACTIVE

FROZEN

BLOCKED

CLOSED

REJECTED

I'd define:

FROZEN

- Account remains open.
- Temporary restriction.
- Transactions are blocked.
- Can be unfrozen.

BLOCKED

- More serious restriction.
- Customer cannot use the account.
- Requires explicit administrative action to unblock.

Then:

ACTIVE

|

+--> FROZEN

| |
| +--> ACTIVE
|
+--> BLOCKED
|
+--> ACTIVE

Don't allow arbitrary status transitions.

7. Your Account PIN needs reconsideration

You said:

Every account has a four digit pin to check balance and confirm transaction.

This is the area I would change most.

A 4-digit PIN is too weak to be the only control for financial transactions.

For your academic project, you can still implement an account PIN because it is a functional requirement, but I recommend:

Account PIN

+

JWT authentication

+

Transaction OTP / MFA

For example:

Check balance

Login

↓

JWT

↓

Select account

↓

Enter 4-digit account PIN



Balance

Transfer

Login



JWT



Transfer details



Account PIN



Email OTP



Transaction confirmed

Your existing MFA requirement in Auth Service makes this particularly appropriate.

8. Don't store the PIN as plaintext

Definitely avoid:

ACCOUNT_PIN = 1234

Instead store a salted hash.

For example:

ACCOUNT_PIN_HASH

PIN_CREATED_AT

PIN_FAILED_ATTEMPTS

PIN_LOCKED_UNTIL

You should also define:

3 or 5 failed attempts

↓

PIN temporarily locked

9. "Switch account type" needs clarification in the domain model

You said:

Switch account type (If one savings account, then change it to current and vice versa)

I would **not make this a simple update**:

UPDATE ACCOUNT

SET ACCOUNT_TYPE = 'CURRENT';

because changing a banking product can have business implications.

For this project, if you must support it, create an explicit workflow:

Savings

↓

Type-change request

↓

Validation

↓

Approved

↓

Account becomes Current

And maintain history:

ACCOUNT_TYPE_HISTORY

HISTORY_ID

ACCOUNT_ID

OLD_TYPE

NEW_TYPE

CHANGED_BY

CHANGED_AT

REASON

Also define what happens when a customer has selected **Both**.

10. Closing an account

Good requirement, but don't make:

DELETE FROM ACCOUNT

Instead:

ACCOUNT.STATUS = CLOSED

and maintain:

CLOSED_AT

CLOSURE_REASON

The account and ledger history should remain.

11. Transaction Service should NOT own the balance

This should be one of your strongest architectural rules.

Transaction Service

```
|  
| "Transfer 10,000"  
v
```

Account Service

```
|  
+--> Debit  
+--> Credit  
+--> Ledger
```

Transaction Service owns:

transaction status

transaction reference

idempotency

transaction history

Account Service owns:

balance

available balance

debit

credit

ledger

This aligns with the general principle already used in the TransactSphere reference, where the account side is responsible for the actual account/ledger operations while transaction processing orchestrates the flow.

12. Internal transfer is a good addition

You have:

internal transfer between self account

I'd explicitly distinguish two transfer types:

SELF_TRANSFER

Example:

Customer C1001

Savings A100

↓ ₹10,000

Current A200

and:

CUSTOMER_TRANSFER

Example:

Customer C1001

Savings A100

↓ ₹10,000

Customer C2002

Savings B100

This will help your Transaction Service.

13. Statement generation

You said:

Able to download account statement

I would put the **statement API in Transaction Service**, because it owns transaction history from the application's perspective.

Example:

GET /transactions/statements

But the actual ledger data remains owned by Account Service.

So:

Transaction Service

|

| OpenFeign

v

Account Service

|

v

Ledger

Then Transaction Service generates:

PDF / CSV

This is cleaner than duplicating ledger data.

14. Notification Service

Your requirement is very clear:

Send email using Google SMTP whenever an event is published from Kafka.

Excellent fit for Kafka.

Any Service

|

| Domain Event

v

Kafka

|

v

Notification Service

|

v

Google SMTP

|

v

Customer Email

Example:

ACCOUNT_APPROVED

TRANSACTION_COMPLETED

TRANSACTION_FAILED

PASSWORD_RESET

LOGIN_ALERT

The reference TransactSphere also deliberately decouples notification delivery from transaction processing using Kafka, which is exactly the pattern I'd retain.

15. Auth Service

Your Auth Service should contain:

Registration

Login

Logout

Password

Forgot Password

MFA / OTP

JWT

Role

Session / Refresh Token

JWT:

```
{  
  "sub": "AUTH_USER_501",  
  "customerId": "C1001",  
  "role": "CUSTOMER"  
}
```

or:

```
{  
  "sub": "AUTH_USER_900",  
  "role": "ADMIN"  
}
```

Use roles:

CUSTOMER

ADMIN

I would **not put the admin's privileges directly into the JWT**. Keep the JWT relatively small and let authorization rules determine what the role can do.

16. Admin audit-log access needs a better model

Your requirement says:

admin must require a passkey from the audit team to access audit logs.

The business requirement is reasonable, but don't implement it as:

Admin enters:

AUDIT_TEAM_PASSKEY = "123456"

because that becomes a shared secret.

I'd implement:

Admin

|

| Request audit access

v

Audit/Notification service

|

| Audit team approval

v

Temporary access token

|

v

Admin

|

| View audit logs

v

Audit data

Or, if your academic requirement specifically says "passkey", make it a **one-time/time-bound approval code** issued by an audit-team user.

For example:

Audit Team approves

↓

PASSKEY valid for 5 minutes

↓

Admin enters passkey

↓

Audit logs unlocked

Also create an audit event for:

ADMIN_AUDIT_ACCESS_REQUESTED

ADMIN_AUDIT_ACCESS_APPROVED

ADMIN_AUDIT_LOG_VIEWED

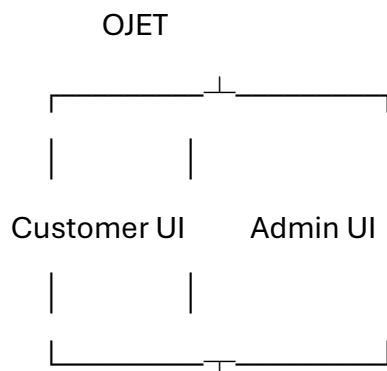
That gives you a nice security story for your project.

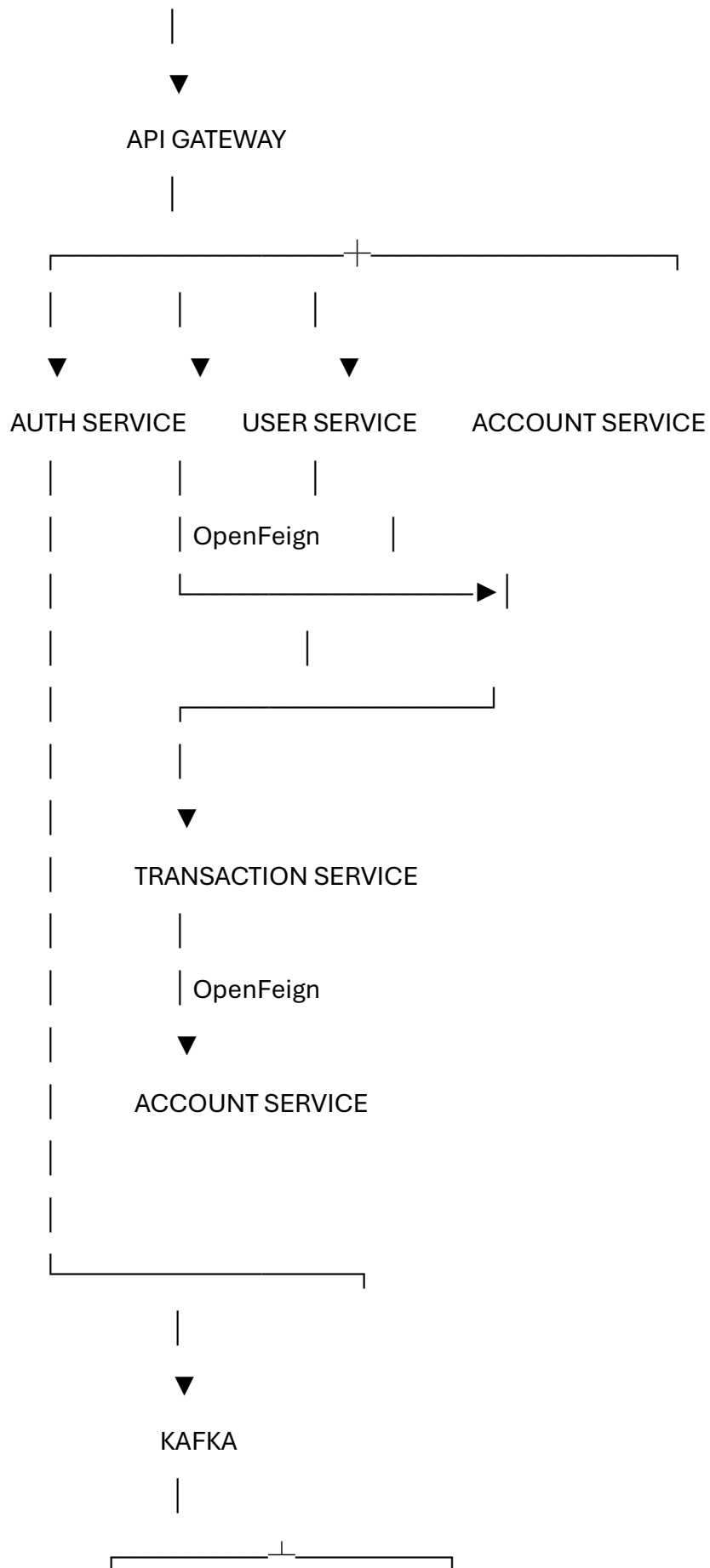
17. Your final responsibilities should look like this

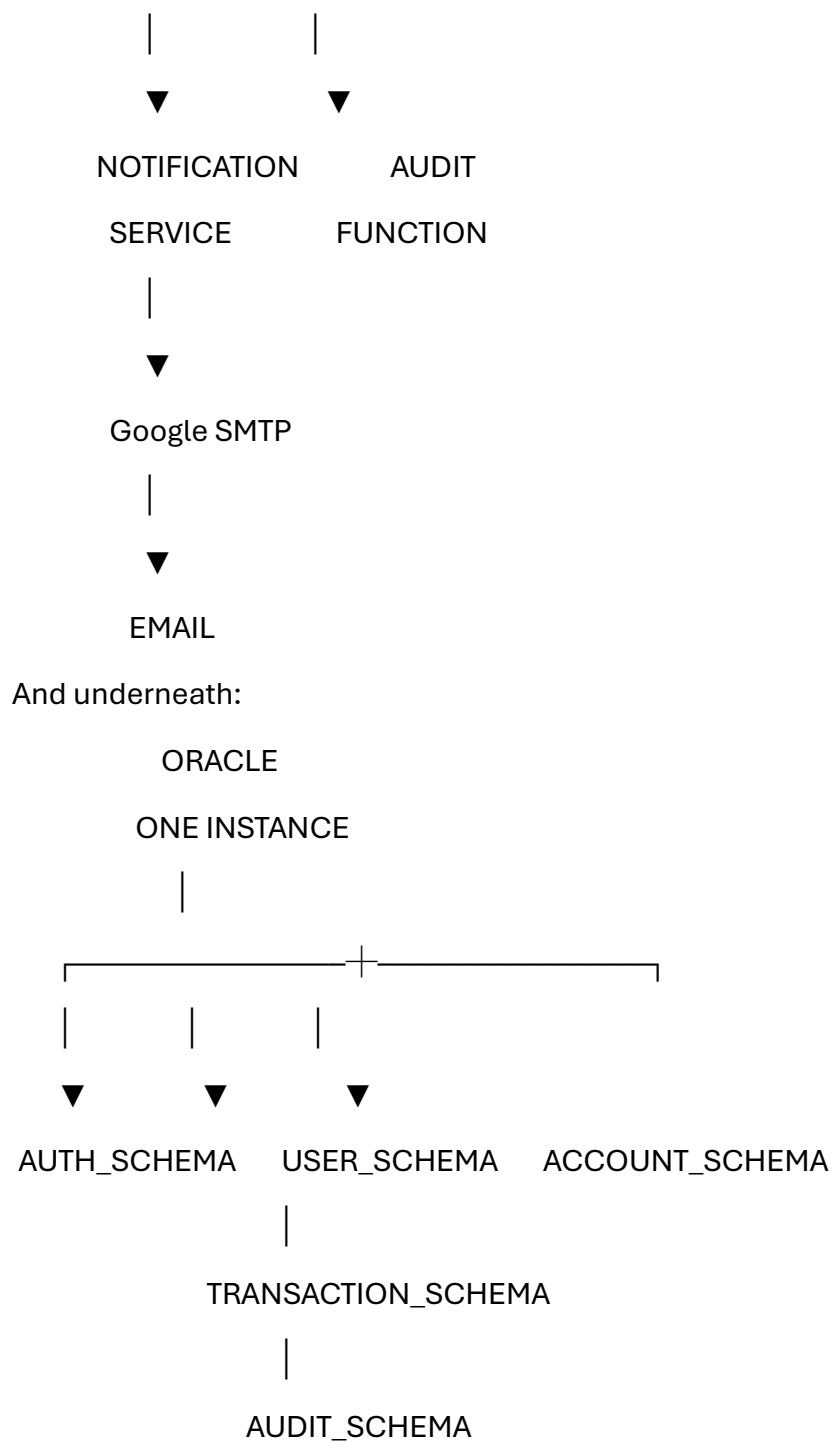
Service	Owns
Auth Service	Registration identity, credentials, login, JWT, MFA, roles
User Service	Customer profile, admin/customer dashboards, customer approval workflow
Account Service	Account creation, account status, balances, PIN, debit/credit, ledger, freeze/block, close
Transaction Service	Transfers, self-transfer, transaction lifecycle, risk, statements
Notification Service	Kafka consumers, email, audit records, audit-access mechanism

This is the version I recommend.

18. The resulting architecture







And underneath:

I would keep **one Oracle instance with five schemas**, as we agreed.

19. One final change to your wording

I would change this:

"When approved the account gets linked with the user email"

to:

"When approved, the account is associated with the customer's CUSTOMER_ID and becomes accessible through the customer's authenticated digital identity."

Email should be the **login/contact identifier**, not the actual account relationship.

Conceptually:

AUTH_USER

|

| CUSTOMER_ID

v

CUSTOMER

|

| CUSTOMER_ID

v

ACCOUNT

not:

EMAIL

|

v

ACCOUNT

That will make your database and service architecture substantially cleaner.

My overall assessment

Your requirements are **good enough to start design**, but I would lock down these six decisions before coding:

1. **User Service owns the dashboard; Account Service owns account operations.**
2. **"Both" = two separate accounts, not one BOTH account.**
3. **Account PIN is hashed and supplemented by MFA/OTP for transactions.**
4. **Transaction Service never directly modifies balances.**
5. **Audit access uses a time-bound/one-time approval mechanism, not a permanent shared passkey.**

6. **Customer ID, not email, is the canonical relationship between identity, customer, and accounts.**

These changes also keep your project close to the useful architectural patterns in **TransactSphere** while adapting its current multi-service/PostgreSQL/Kubernetes implementation to your required **five services, Oracle, OJET, Eureka, Kafka, OpenFeign, and Podman.**